

Air Hockey Lab - Conversation Transcript (Redacted)

Complete conversation between Aadhavan and Claude, in order. Aadhavan's prompts are shown under "AADHAVAN"; Claude's replies under "CLAUDE". Text is reproduced verbatim except for redactions of local system paths and file names, marked **[REDACTED...]**; GitHub links are left intact. Two layout-only adjustments: attached files are listed by name in a bracketed line instead of being reprinted (their contents appear inside the code blocks of the replies), and code lines longer than the page width are wrapped onto a continuation line.

AADHAVAN

you are a senior software engineer dwelling into vibe coding territory
your task now is to give me detailed instructions along with exactly what to do and what to capture , as part of a college assignment

the task basically involves me cloning a provided repo, fixing bugs and changes and what not (given under readme) and perform the tasks

you DO NOT HAVE TO MAKE A REPO, OR ANYTHING GITHUB RELATED, YOUR WORK IS PURELY TO GIVE ME CHANGES TO MAKE IN CODE AND OTHER BASIC INSTRUCTIONS

these are instructions provided in the document shared to me

1. Go to the repo assigned to you
2. Go through the ReadMe file to understand the deliverables
3. Clone or Fork the repo
4. Run the Python based code (A game/ A Puzzle)
5. Record 10 seconds of video before making any changes.
6. Write prompts and fix the broken code
7. Write prompts to add all the features listed in the ReadMe file of the repo one by one.
8. Record another 10 seconds of video again after making all the changes.
9. Push the code to your repo.

this is the readme given for my submission

Air Hockey Lab

This project is a single-topic Air Hockey game using Pygame. It introduces students to angle-based collision physics, scoring, round timing, and post-goal state reset, using a small, readable object-oriented codebase.

What's Provided

A working Air Hockey game with:

- * A player paddle (blue, left side) moved with the arrow keys, and a computer-controlled paddle on the right
- * A puck that bounces off the top and bottom walls and off both paddles
- * Goals at the center of each end wall

It has one deliberate bug and three features left for you to build. You are expected to analyze, interact with an AI assistant, and complete/fix the game to make it fully functional and more interesting.

Use an LLM (e.g. ChatGPT or Claude) as your debugging and pair-programming partner for this lab.

Getting Started

Setup

1. Make sure you have Python 3.10+ installed.
2. Install dependencies:

```
...
pip install -r requirements.txt
...
```

3. Run the game:

```
...
python main.py
...
```

Controls: Arrow keys move your paddle (blue, left side) within your own half of the table. Press R at any time to restart.

Tasks to Complete

Each task must be completed using an iterative process involving LLM suggestions and your critical code review.

Task 1: Fix puck-paddle collision

The puck does not consistently respond correctly when it collides with a paddle, particularly at higher speeds or when it approaches at an angle. Fix the collision logic so the puck reliably rebounds from the paddles at different speeds and angles without passing through or getting stuck inside them.

Task 2: Implement match scoring

Add a scoring system where a player earns a point when the puck enters the opponent's goal. Display both players' scores during play, and determine the winner based on the final score.

Task 3: Implement a 30-second match timer

Add a 30-second countdown timer to the match. Display the remaining time during gameplay. When the timer reaches zero, stop the match and determine the result based on the scores. If both scores are equal, display a draw.

Task 4: Implement puck reset after scoring

After a goal is scored, reset the puck to the center of the table and prepare it for the next point. Make sure the puck's velocity, direction, and any other relevant state are correctly reset so each new point starts cleanly.

Expected Behavior

- * The puck bounces realistically off walls and paddles at any angle, and never gets stuck vibrating inside a paddle.
- * A point is scored only when the puck fully passes through the goal gap - hitting the wall elsewhere bounces the puck back normally.
- * After every goal, the puck immediately continues play from the center - it should never sit motionless.
- * The match runs for exactly 30 seconds. When time runs out, the match ends and shows the correct winner - or "Draw" if the score is tied.

Folder Structure

```
...
air-hockey/
├── main.py
├── requirements.txt
├── game/
│   ├── game_engine.py
│   ├── puck.py
│   ├── paddle.py
│   ├── collisions.py
│   ├── ai.py
│   └── renderer.py
└── README.md
...
```

Submission Checklist

Submission is only the following three things:

- * A 10-second video of gameplay before your changes, showing the bug/broken behavior
- * A 10-second video of gameplay after your changes, showing the bug fixed and the new features working
- * The Chat/LLM used page link, with the complete chat history

acknowledge, lets get the prerequisites done first

also for the video recording, im on mac, i have built in screen recording

do not assume anything without clarifying or verifying first, give short concise informative instructions

ill clone the repo, then ill share the individual file contents

give me exact and all terminal commands

CLAUDE

Before we start, a few things to check, because your assignment steps have a gap.

Step 3 says clone or fork, but step 9 says push to your repo. If the assigned repo isn't yours (for example, it belongs to your instructor or a template), a plain clone won't let you push. Fork it first on GitHub, then clone your fork. If it's already under your account, clone it directly. Tell me which case it is. Also, the checklist only requires the two videos and the chat link, so confirm with your instructor whether the push is actually graded.

Since the checklist needs the complete chat history, do the entire assignment in this one chat and share its link at the end. Don't start new chats.

Check your tools first:

```
python3 --version
git --version
```

You need Python 3.10 or higher. If git is missing, macOS will prompt you to install the command line tools, or run `xcode-select --install`. On Mac the README's `python` and `pip` commands often don't exist; use `python3` and `pip3`, inside a virtual environment. Note that pygame sometimes has no prebuilt package for the newest Python releases, so if the install fails, tell me your exact Python version and the error.

Clone and set up (replace the placeholders with your own URL and folder name):

```
git clone https://github.com/<your-username>/<repo-name>.git
cd <repo-name>
git remote -v
ls -R
cat requirements.txt
python3 -m venv venv
source venv/bin/activate
pip install -r requirements.txt
python -c "import pygame; print(pygame.version.ver)"
python main.py
```

`git remote -v` should show your own account in the URL. If it shows someone else's, you won't be able to push. Run `source venv/bin/activate` again in any new terminal window before running the game.

Before-video: with the game running, press Cmd+Shift+5, choose "Record Selected Portion", and drag over the game window. Under Options, set the save location to Desktop and turn the microphone off. Click Record, then click the game window, because pygame only reads the arrow keys when it has focus. Play for about 10 seconds, deliberately hitting the puck fast and at angles to expose the paddle collision bug (puck passing through or sticking). Stop with the stop icon in the menu bar or Cmd+Control+Esc. If the clip runs longer than 10 seconds, open it in QuickTime and trim with

Cmd+T. Don't edit any code until the video is saved.

Once the game runs, tell me your repo situation (fork or already yours), your Python version, and what the contents of `requirements.txt` are. Then share the files, starting with `game/collisions.py`, `game/puck.py`, `game/paddle.py`, and `game/game_engine.py`.

AADHAVAN

before i do all this

instead of pushing to that repo, i can clone , do the changes and push to my preexisting repo under a different origin can't i?

my preexisting repo has previous lab folders as well , i can make a new folder as Lab_04 and push to this ?

CLAUDE

Yes, but not by changing the clone's origin. If you repoint the clone's `origin` at your existing repo and push, you'd be pushing the assigned repo's entire history into a repo with unrelated history. The push would be rejected, or you'd have to merge unrelated histories, and the files would land at the repo root instead of in `Lab_04`.

The clean way is to do all the work in the cloned assigned repo (which you only need locally), and when you're done, copy the project files into a `Lab_04` folder inside a clone of your existing repo, then commit and push from there. Copy without the `.git` folder, otherwise you'd nest one repo inside another and Git would treat `Lab_04` as an embedded repo instead of normal files. Also leave out `venv` and `__pycache__`.

When you're finished with the assignment, run this from wherever you keep your projects (adjust the names):

```
git clone https://github.com/<your-username>/<existing-repo>.git
cd <existing-repo>
git branch --show-current
mkdir Lab_04
rsync -av --exclude '.git' --exclude 'venv' --exclude '__pycache__' --exclude '.DS_Store' ..
    /<assigned-repo-folder>/ Lab_04/
git status
git add Lab_04
git commit -m "Add Lab_04: Air Hockey"
git push
```

Skip the first `git clone` line if you already have your existing repo locally, and run `git pull` inside it first. The trailing slashes on `../<assigned-repo-folder>/` and `Lab_04/` matter: they copy the folder's contents rather than the folder itself. Check `git status` before committing to confirm only `Lab_04` files are staged, with no `venv`.

One caveat: your instructor may expect the link to the specific assigned or forked repo rather than a folder in your own. The checklist you pasted only asks for the two videos and the chat link, so a push may not be graded at all, but it's worth a quick confirmation so you don't lose marks over it.

This means the earlier fork-versus-clone question no longer matters: a plain clone of the assigned repo is enough. Go ahead with the setup steps from my previous message, skipping the `git remote -v` check, and send me your Python version, the `requirements.txt` contents, and the file contents when ready.

AADHAVAN

this isn't consise and informative, leave it , its alright, now you now the situation, constrains and deliverables, lets start from absolute ground zero , i made a new folder in my mac file system , lets start now

this is the assigned url

https://github.com/SETAPESU26/02_air_hockey/tree/main

this is my url

<https://github.com/paya5am/Software-Engineering-Lab>

this has previous lab submissions under different Lab01 - lab03 folders, pushed from my system using git

this is just for your information

CLAUDE

Open Terminal (Cmd+Space, type "Terminal"). Run these one at a time and send me the output of each block.

Step 1, check tools:

```
python3 --version
git --version
```

Python must be 3.10 or higher. If git isn't installed, macOS will offer to install the command line tools; accept it and rerun.

Step 2, go to your new folder and clone. Replace `<path>` with the folder you made (you can drag the folder from Finder into Terminal after typing `cd`):

```
cd <path>
git clone https://github.com/SETAPESU26/02_air_hockey.git
cd 02_air_hockey
ls -R
cat requirements.txt
```

Step 3, virtual environment and dependencies:

```
python3 -m venv venv
source venv/bin/activate
pip install -r requirements.txt
python -c "import pygame; print(pygame.version.ver)"
```

Step 4, run the game (don't record yet, just confirm it launches):

```
python main.py
```

Send me your Python version, the `ls -R` output, the `requirements.txt` contents, the pygame version, and whether the game window opened. If any command errors, paste the full error.

Do not edit any code and do not record the "before" video until I've seen these results. I'll then give you the exact recording procedure and the bug to trigger, so the clip shows it clearly.

AADHAVAN

step 3 has an error,

requirements is under another subdirectory

so readjust for future references

this is the file structure

```
ls
```

```
air-hockey README.md venv
```

```
(venv) aadhavan@host 02_air_hockey % cd air-hockey
```

```
(venv) aadhavan@host air-hockey % ls
```

```
zsh: command not found: ls
```

```
(venv) aadhavan@host air-hockey % ls
```

```
game main.py requirements.txt
```

CLAUDE

Understood. The project root is `02_air_hockey/air-hockey`, and the venv sits one level above it. That's fine, and it will make the final copy easier since only `air-hockey/` gets copied. The `(venv)` prefix shows it's active. Run this from your current location, inside `air-hockey`:

```
cat requirements.txt
pip install -r requirements.txt
python -c "import pygame; print(pygame.version.ver)"
python --version
ls game
python main.py
```

Send me the output of the `cat`, both version lines, and the `ls game` listing (it should show the files from the README structure), and tell me whether the game window opened. If anything errors, paste the full error.

From here on, every command assumes you're inside `02_air_hockey/air-hockey` with the venv active. If you open a new terminal window, run `cd <path>/02_air_hockey/air-hockey` and then `source ../venv/bin/activate` first.

Still no code edits and no recording until I've seen the results.

AADHAVAN

i ran it, the game ran,

```
cat requirements.txt
```

```
pip install -r requirements.txt
```

```
python -c "import pygame; print(pygame.version.ver)"
```

```
python --version
```

```
ls game
```

```
python main.py
```

```
pygame>=2.5.0
```

```
Requirement already satisfied: pygame>=2.5.0 in [REDACTED
```

```
PATH]/venv/lib/python3.9/site-packages (from -r requirements.txt (line 1)) (2.6.1)
```

```
WARNING: You are using pip version 21.2.4; however, version 26.0.1 is available.
```

```
You should consider upgrading via the '[REDACTED PATH]/venv/bin/python3 -m pip install --upgrade pip' command.
```

```
pygame 2.6.1 (SDL 2.28.4, Python 3.9.6)
```

```
Hello from the pygame community. https://www.pygame.org/contribute.html
```

```
2.6.1
```

```
Python 3.9.6
```

```
__init__.py ai.py collisions.py game_engine.py paddle.py puck.py renderer.py
```

```
pygame 2.6.1 (SDL 2.28.4, Python 3.9.6)
```

```
Hello from the pygame community. https://www.pygame.org/contribute.html
```

```
(venv) aadhavan@host air-hockey %
```

```
now lets fix the bugs
```

```
this is readme, for reference
```

```
# Air Hockey Lab
```

This project is a single-topic Air Hockey game using `**Pygame**`. It introduces students to angle-based collision physics, scoring, round timing, and post-goal state reset, using a small, readable object-oriented codebase.

```
---
```

```
## What's Provided
```

A working Air Hockey game with:

- A player paddle (blue, left side) moved with the arrow keys, and a computer-controlled paddle on the right
- A puck that bounces off the top and bottom walls and off both paddles
- Goals at the center of each end wall

It has **one deliberate bug** and **three features** left for you to build. You are expected to **analyze**, **interact with an AI assistant**, and **complete/fix** the game to make it fully functional and more interesting.

Use an LLM (e.g. ChatGPT or Claude) as your debugging and pair-programming partner for this lab.

Getting Started

Setup

1. Make sure you have Python 3.10+ installed.
2. Install dependencies:

```
```bash
pip install -r requirements.txt
```
```

3. Run the game:

```
```bash
python main.py
```
```

Controls: Arrow keys move your paddle (blue, left side) within your own half of the table. Press R at any time to restart.

Tasks to Complete

Each task must be completed using an iterative process involving LLM suggestions and your critical code review.

Task 1: Fix puck–paddle collision

- > The puck does not consistently respond correctly when it collides
- > with a paddle, particularly at higher speeds or when it approaches
- > at an angle. Fix the collision logic so the puck reliably rebounds
- > from the paddles at different speeds and angles without passing
- > through or getting stuck inside them.

Task 2: Implement match scoring

- > Add a scoring system where a player earns a point when the puck
- > enters the opponent's goal. Display both players' scores during
- > play, and determine the winner based on the final score.

Task 3: Implement a 30-second match timer

- > Add a 30-second countdown timer to the match. Display the remaining
- > time during gameplay. When the timer reaches zero, stop the match
- > and determine the result based on the scores. If both scores are
- > equal, display a draw.

Task 4: Implement puck reset after scoring

- > After a goal is scored, reset the puck to the center of the table
- > and prepare it for the next point. Make sure the puck's velocity, direction, and any other relevant state are correctly reset so each
- > new point starts cleanly.

Expected Behavior

- The puck bounces realistically off walls and paddles at any angle, and never gets stuck vibrating inside a paddle.
- A point is scored only when the puck fully passes through the goal gap - hitting the wall elsewhere bounces the puck back normally.
- After every goal, the puck immediately continues play from the center - it should never sit motionless.
- The match runs for exactly 30 seconds. When time runs out, the match ends and shows the correct winner - or "Draw" if the score is tied.

Folder Structure

...

```
air-hockey/
├── main.py
├── requirements.txt
├── game/
│   ├── game_engine.py
│   ├── puck.py
│   ├── paddle.py
│   ├── collisions.py
│   ├── ai.py
│   └── renderer.py
└── README.md
```

...

Submission Checklist

Submission is only the following three things:

- [] A 10-second video of gameplay **before** your changes, showing the bug/broken behavior
 - [] A 10-second video of gameplay **after** your changes, showing the bug fixed and the new features working
 - [] The Chat/LLM used page link, with the complete chat history
- acknowledge and reply in short do not make assumptions

CLAUDE

Acknowledged. Two notes:

1. Python is 3.9.6, the README asks for 3.10+. The game launched, so it works for now. If we hit a syntax error involving `X | Y` type hints or `match` statements, that's the cause, and I'll tell you how to handle it.
2. Record the "before" video first, since the checklist needs the broken behavior on camera. Don't edit anything yet.

Recording:

1. Run `python main.py`.
2. Press Cmd+Shift+5, choose "Record Selected Portion", drag over the game window. In Options, turn the microphone off.
3. Click Record, then click the game window so it has keyboard focus.
4. Play ~10 seconds. Hit the puck fast and at angles, and push it into the AI paddle from the side, to expose passing-through or sticking.
5. Stop with Cmd+Control+Esc. If the clip is longer than 10 seconds, trim it in QuickTime with Cmd+T.

After that, send me the full contents of these files, each labeled with its filename:

`game/collisions.py`, `game/puck.py`, `game/paddle.py`, `game/game_engine.py`. I'll ask for `renderer.py`, `ai.py` and `main.py` when needed. I won't propose any fix until I've read the actual code.

Also tell me what you observed in the game (puck behavior, whether the AI paddle moves, anything that looked wrong), since I can't see the screen.

AADHAVAN

[Attached files: `__init__.py`, `ai.py`, `collisions.py`, `game_engine.py`, `paddle.py`, `puck.py`, `renderer.py`, `main.py`]

i shall record the screen, after a goal is hit, puck sits at center, nothing happens, and slight jittering when my paddle hits the puck, this is the bug im guessing, also fixing logic is mentioned in the four tasks , i shall upload the game files now, so that you can crossverify once with the code itself
do not generate anything yet, lets verify before moving on
keep replies short

CLAUDE

Your observations match the code, but you've attributed them to the wrong tasks.

The jitter is the deliberate bug (Task 1). `handle_paddle_collision` in `collisions.py` only does `puck.vx = -puck.vx` when the circles overlap. It never pushes the puck out of the paddle, so the overlap persists next frame and the velocity flips again, which is the vibrating or sticking. It also ignores where the puck hit the paddle (a hit from the top or bottom still just reverses vx, so angles are wrong) and ignores paddle movement.

The frozen puck is not that bug. It is the incomplete reset (Task 4): `_reset_puck` in `game_engine.py` sets vx and vy to 0 and never relaunches, although `_launch_puck` already exists.

Other things I found that you didn't mention:

1. `_handle_goals` triggers as soon as the puck's edge crosses the wall line, but the README says a goal counts only when the puck fully passes through the gap. This needs a decision before Task 2.
2. The README says R restarts the game, but `main.py` has no key handling for R, so it currently does nothing.
3. No scoring or timer state exists anywhere. `draw()` receives `font` but never uses it, and `renderer.py` already has `draw_text` and `draw_banner` we can use.
4. Python 3.9.6 is fine so far, since nothing in the code uses 3.10-only syntax.

Have you already recorded and saved the "before" video? Once you confirm, I'll propose the Task 1 fix first.

AADHAVAN

before video has been recorded

lets proceed

faculty mentioned i have to make a minimum of four commits, so plan accordingly

i have to commit to my software-engineering-lab repo , by making a new folder named lab-4 and then the game files

give me complete file after making changes into original given file, just put comments to signify that its been added

my software engineering repo is up to date like this,

```
cd [REDACTED PATH]
```

```
aadhavan@host software % ls
```

```
[REDACTED FILE]    lab                [REDACTED]                [REDACTED]
```

```
aadhavan@host software % cd lab
```

```
aadhavan@host lab % ls
```

```
Lab_01            Lab_02            Lab_03            Lab_04            README.md        [REDACTED]
```

```
aadhavan@host lab % git status
```

On branch main

Untracked files:

(use "git add <file>..." to include in what will be committed)

```
.gitignore
```

```
Lab_03/[REDACTED FILE]
```

```
Lab_03/[REDACTED FILE]
```

```
Lab_03/[REDACTED FILE]
```

```
Lab_03/[REDACTED FILE]
```

```
Lab_03/[REDACTED FILE]
```

```
Lab_03/[REDACTED FILE]
```

```
Lab_04/
```

```
[REDACTED]/
```

nothing added to commit but untracked files present (use "git add" to track)

```
aadhavan@host lab %
```

the above mentioned files that have not been added, need not be , give changes appropriately

my mistake stating earlier stating lab-4, follow as Lab_04

this is the git and directory structure for reference

```
cd [REDACTED PATH]
```

```
git remote -v
```

```
ls -a Lab_04/02_air_hockey
```

```
origin    https://github.com/paya5am/Software-Engineering-Lab.git (fetch)
```

```
origin    https://github.com/paya5am/Software-Engineering-Lab.git (push)
```

```
.          ..          .DS_Store    .git          .gitignore    air-hockey    README.md
```

```
venv
```

```
aadhavan@host lab % git remote -v
```

```
origin    https://github.com/paya5am/Software-Engineering-Lab.git (fetch)
```

```
origin    https://github.com/paya5am/Software-Engineering-Lab.git (push)
```

```
aadhavan@host lab % ls -a Lab_04/02_air_hockey
```

```
.          ..          .DS_Store    .git          .gitignore    air-hockey    README.md
```

```
venv
```

```
aadhavan@host lab % ls
```

```
Lab_01            Lab_02            Lab_03            Lab_04            README.md        [REDACTED]
```

```
aadhavan@host lab % cd Lab_04
```

```
aadhavan@host Lab_04 % ls
```

```
02_air_hockey
```

```
aadhavan@host Lab_04 % cd 02_air_hockey
```

```
aadhavan@host 02_air_hockey % ls
```

```
air-hockey    README.md    venv
```

```
aadhavan@host 02_air_hockey % cd air-hockey
aadhavan@host air-hockey % ls
game                main.py             requirements.txt
aadhavan@host air-hockey %
```

CLAUDE

Order matters here: commit the untouched starter code first (that's commit 1), then apply the Task 1 files as commit 2.

Your clone left a `.git` folder inside `Lab_04/02_air_hockey`. If you commit `Lab_04` with it present, your outer repo records `02_air_hockey` as an embedded repo and none of the game files get committed. It only holds the assigned repo's clone metadata, so delete it. The structure stays `Lab_04/02_air_hockey/air-hockey/...`, so tell me if you want it flattened. Run:

```
cd [REDACTED PATH]
rm -rf Lab_04/02_air_hockey/.git
cat Lab_04/02_air_hockey/.gitignore
git add -n Lab_04
```

`git add -n` is a dry run that only lists what would be staged. If any line contains `venv` or `__pycache__`, or you see `.DS_Store`, stop and paste the output to me. Otherwise continue. Never use `git add .`, since it would stage your other untracked files.

```
git add Lab_04
git status --short
git commit -m "Lab_04: add starter Air Hockey code"
```

`git status --short` should show only `Lab_04` files as staged, with the other untracked items still untracked. Do not push yet.

Now Task 1. The fix has four parts: push the puck out of the paddle so overlap can't persist, reflect the velocity about the contact normal so the angle is correct, add the paddle's own velocity so hits feel physical, and cap the puck speed at 14. That cap keeps it below the 40 px combined radius per frame, which is what rules out tunneling. It needed paddle velocity tracking, so three files change. Replace them fully, inside `air-hockey`.

`game/paddle.py`:

```
"""
Paddle: a player's or the computer's mallet, confined to their own half
of the table.
"""

class Paddle:
    def __init__(self, x, y, radius, min_x, max_x, min_y, max_y):
        self.x = x
        self.y = y
        self.radius = radius
        self.min_x = min_x
        self.max_x = max_x
        self.min_y = min_y
        self.max_y = max_y

        # ADDED (Task 1): velocity tracking so collisions can use paddle motion
        self.prev_x = x
        self.prev_y = y
        self.vx = 0.0
        self.vy = 0.0

    def clamp(self):
        self.x = max(self.min_x, min(self.max_x, self.x))
```

```

        self.y = max(self.min_y, min(self.max_y, self.y))

def move_by(self, dx, dy):
    self.x += dx
    self.y += dy
    self.clamp()

def move_to(self, x, y):
    self.x = x
    self.y = y
    self.clamp()

# ADDED (Task 1): call once per frame after all paddle movement is done
def update_velocity(self):
    self.vx = self.x - self.prev_x
    self.vy = self.y - self.prev_y
    self.prev_x = self.x
    self.prev_y = self.y

```

game/collisions.py:

```

"""
collisions: puck-vs-paddle collision handling.
"""

import math # ADDED (Task 1)

MAX_PUCK_SPEED = 14.0 # ADDED (Task 1): keeps per-frame movement below combined radii (no tunneling)

def handle_paddle_collision(puck, paddle, min_y=None, max_y=None):
    """
    If the puck overlaps the paddle, push it out and bounce it off.
    min_y / max_y (optional) are the limits for the puck's centre so it
    can be slid sideways instead of being pushed into a top/bottom wall.
    Returns True if a collision was handled this frame.
    """
    dx = puck.x - paddle.x
    dy = puck.y - paddle.y
    distance = math.hypot(dx, dy) # CHANGED (Task 1)
    min_dist = puck.radius + paddle.radius

    if distance >= min_dist:
        return False

    # ADDED (Task 1): collision normal, pointing from paddle centre to puck centre
    if distance > 1e-6:
        nx, ny = dx / distance, dy / distance
    else:
        speed = math.hypot(puck.vx, puck.vy)
        if speed > 1e-6:
            nx, ny = -puck.vx / speed, -puck.vy / speed
        else:
            nx, ny = 0.0, -1.0

    # ADDED (Task 1): positional correction - move puck fully out of the paddle
    puck.x = paddle.x + nx * min_dist
    puck.y = paddle.y + ny * min_dist

    # ADDED (Task 1): if that pushed the puck into a top/bottom wall, slide it
    # along the paddle's edge instead so it can't get pinned and vibrate
    if min_y is not None and max_y is not None:
        clamped_y = max(min_y, min(max_y, puck.y))
        if clamped_y != puck.y:
            puck.y = clamped_y
            dy = puck.y - paddle.y

```

```

        side = 1.0 if puck.x >= paddle.x else -1.0
        puck.x = paddle.x + side * math.sqrt(max(min_dist ** 2 - dy ** 2, 0.0))
        nx = (puck.x - paddle.x) / min_dist
        ny = dy / min_dist

# ADDED (Task 1): reflect velocity about the normal, relative to the moving paddle.
# Only when approaching (vn < 0); if already separating, leave velocity alone.
rvx = puck.vx - paddle.vx
rvy = puck.vy - paddle.vy
vn = rvx * nx + rvy * ny
if vn < 0:
    rvx -= 2 * vn * nx
    rvy -= 2 * vn * ny
    puck.vx = rvx + paddle.vx
    puck.vy = rvy + paddle.vy

# ADDED (Task 1): speed cap
speed = math.hypot(puck.vx, puck.vy)
if speed > MAX_PUCK_SPEED:
    scale = MAX_PUCK_SPEED / speed
    puck.vx *= scale
    puck.vy *= scale

return True

```

game/game_engine.py:

```

"""
GameEngine: owns the puck, both paddles, and the computer AI, and runs
one frame's worth of game logic.

Starter version: the puck bounces around and paddles can hit it, but
there is no scoring, no match timer, and the reset that happens after
a goal is incomplete. That's what Tasks 2-4 fix/add.
"""

import random

from game.puck import Puck
from game.paddle import Paddle
from game.ai import ComputerAI
from game.collisions import handle_paddle_collision
from game.renderer import WIDTH, HEIGHT, MARGIN, GOAL_TOP, GOAL_BOTTOM

PLAYER_SPEED = 6
PUCK_RADIUS = 12
PADDLE_RADIUS = 28
INITIAL_PUCK_SPEED = 4.5

class GameEngine:
    def __init__(self):
        self.puck = Puck(WIDTH / 2, HEIGHT / 2, PUCK_RADIUS)
        self._launch_puck()

        self.player = Paddle(
            x=WIDTH * 0.15, y=HEIGHT / 2, radius=PADDLE_RADIUS,
            min_x=MARGIN + PADDLE_RADIUS, max_x=WIDTH / 2 - PADDLE_RADIUS,
            min_y=MARGIN + PADDLE_RADIUS, max_y=HEIGHT - MARGIN - PADDLE_RADIUS,
        )
        self.computer = Paddle(
            x=WIDTH * 0.85, y=HEIGHT / 2, radius=PADDLE_RADIUS,
            min_x=WIDTH / 2 + PADDLE_RADIUS, max_x=WIDTH - MARGIN - PADDLE_RADIUS,
            min_y=MARGIN + PADDLE_RADIUS, max_y=HEIGHT - MARGIN - PADDLE_RADIUS,
        )
        self.ai = ComputerAI()

```

```

def _launch_puck(self):
    angle_choices = [0.3, 0.6, -0.3, -0.6]
    direction = random.choice([-1, 1])
    vy_factor = random.choice(angle_choices)
    self.puck.vx = INITIAL_PUCK_SPEED * direction
    self.puck.vy = INITIAL_PUCK_SPEED * vy_factor

def handle_input(self, keys_pressed):
    import pygame
    dx = dy = 0
    if keys_pressed[pygame.K_UP]:
        dy -= PLAYER_SPEED
    if keys_pressed[pygame.K_DOWN]:
        dy += PLAYER_SPEED
    if keys_pressed[pygame.K_LEFT]:
        dx -= PLAYER_SPEED
    if keys_pressed[pygame.K_RIGHT]:
        dx += PLAYER_SPEED
    self.player.move_by(dx, dy)

def update(self):
    self.ai.update(self.computer, self.puck)

    # ADDED (Task 1): record this frame's paddle velocities before colliding
    self.player.update_velocity()
    self.computer.update_velocity()

    self.puck.move()
    self.puck.bounce_off_walls(HEIGHT, MARGIN)

    # CHANGED (Task 1): pass the puck-centre y limits to the collision handler
    min_y = MARGIN + PUCK_RADIUS
    max_y = HEIGHT - MARGIN - PUCK_RADIUS
    handle_paddle_collision(self.puck, self.player, min_y, max_y)
    handle_paddle_collision(self.puck, self.computer, min_y, max_y)

    self._handle_goals()

def _handle_goals(self):
    if self.puck.x - self.puck.radius < MARGIN:
        if GOAL_TOP < self.puck.y < GOAL_BOTTOM:
            self._reset_puck()
        else:
            self.puck.x = MARGIN + self.puck.radius
            self.puck.vx = -self.puck.vx
    elif self.puck.x + self.puck.radius > WIDTH - MARGIN:
        if GOAL_TOP < self.puck.y < GOAL_BOTTOM:
            self._reset_puck()
        else:
            self.puck.x = WIDTH - MARGIN - self.puck.radius
            self.puck.vx = -self.puck.vx

def _reset_puck(self):
    self.puck.x, self.puck.y = WIDTH / 2, HEIGHT / 2
    self.puck.vx = 0
    self.puck.vy = 0

def draw(self, surface, font):
    from game import renderer
    renderer.draw_table(surface)
    renderer.draw_paddle(surface, self.player, renderer.COLOR_PLAYER)
    renderer.draw_paddle(surface, self.computer, renderer.COLOR_COMPUTER)
    renderer.draw_puck(surface, self.puck)

```

Run `python main.py` and test: hit the puck from the top, bottom, and side; sweep the paddle into a moving puck; and try pinning the puck against the top or bottom wall with your paddle. Expect no

jitter, correct angles, and faster shots when your paddle is moving. The puck still freezes after a goal, since that's Task 4. One known limit: the corner-slide fix only covers the top and bottom walls. Squeezing the puck between your paddle and the left end wall outside the goal gap can still jitter, because the paddle's minimum x leaves only 16 px of gap there. Tell me if you see that.

If it behaves, commit:

```
cd [REDACTED PATH]
git add Lab_04/02_air_hockey/air-hockey/game
git commit -m "Task 1: fix puck-paddle collision"
```

Report what you see when you test.

AADHAVAN

okay, good, changes made, git upto speed, changes observed, should i push before moving onto next task? , after that lets proceed to next task, give exact full file replacements (as mentioned before) along with that the files are fixing, exact git adding and committing steps as well

CLAUDE

Yes, push now. It backs up your work and it doesn't affect the remaining commits:

```
cd [REDACTED PATH]
git push origin main
```

If it's rejected, paste the message. You now have 2 commits. Tasks 2, 3 and 4 will each be their own commit, giving 5 in total.

Task 2 (scoring) only changes `game/game_engine.py`. I've made one interpretation call: the README says a goal counts only when the puck fully passes through the gap, so I changed `_handle_goals` to score when the puck's whole body is past the goal line, and only if it fits inside the gap. Otherwise it bounces off the wall as before. A puck entering the left goal scores for the computer, and the right goal scores for you. I also added `winner_text()` and a `match_over` flag, but nothing sets the flag until Task 3, so no winner banner will appear yet.

Replace the whole file with:

```
"""
GameEngine: owns the puck, both paddles, and the computer AI, and runs
one frame's worth of game logic.

Starter version: the puck bounces around and paddles can hit it, but
there is no scoring, no match timer, and the reset that happens after
a goal is incomplete. That's what Tasks 2-4 fix/add.
"""

import random

from game.puck import Puck
from game.paddle import Paddle
from game.ai import ComputerAI
from game.collisions import handle_paddle_collision
from game.renderer import WIDTH, HEIGHT, MARGIN, GOAL_TOP, GOAL_BOTTOM

PLAYER_SPEED = 6
PUCK_RADIUS = 12
PADDLE_RADIUS = 28
INITIAL_PUCK_SPEED = 4.5

class GameEngine:
    def __init__(self):
        self.puck = Puck(WIDTH / 2, HEIGHT / 2, PUCK_RADIUS)
```

```

self._launch_puck()

self.player = Paddle(
    x=WIDTH * 0.15, y=HEIGHT / 2, radius=PADDLE_RADIUS,
    min_x=MARGIN + PADDLE_RADIUS, max_x=WIDTH / 2 - PADDLE_RADIUS,
    min_y=MARGIN + PADDLE_RADIUS, max_y=HEIGHT - MARGIN - PADDLE_RADIUS,
)
self.computer = Paddle(
    x=WIDTH * 0.85, y=HEIGHT / 2, radius=PADDLE_RADIUS,
    min_x=WIDTH / 2 + PADDLE_RADIUS, max_x=WIDTH - MARGIN - PADDLE_RADIUS,
    min_y=MARGIN + PADDLE_RADIUS, max_y=HEIGHT - MARGIN - PADDLE_RADIUS,
)
self.ai = ComputerAI()

# ADDED (Task 2): match scores
self.player_score = 0
self.computer_score = 0
# ADDED (Task 2): set to True by the match timer in Task 3
self.match_over = False

def _launch_puck(self):
    angle_choices = [0.3, 0.6, -0.3, -0.6]
    direction = random.choice([-1, 1])
    vy_factor = random.choice(angle_choices)
    self.puck.vx = INITIAL_PUCK_SPEED * direction
    self.puck.vy = INITIAL_PUCK_SPEED * vy_factor

def handle_input(self, keys_pressed):
    import pygame
    dx = dy = 0
    if keys_pressed[pygame.K_UP]:
        dy -= PLAYER_SPEED
    if keys_pressed[pygame.K_DOWN]:
        dy += PLAYER_SPEED
    if keys_pressed[pygame.K_LEFT]:
        dx -= PLAYER_SPEED
    if keys_pressed[pygame.K_RIGHT]:
        dx += PLAYER_SPEED
    self.player.move_by(dx, dy)

def update(self):
    self.ai.update(self.computer, self.puck)

    # ADDED (Task 1): record this frame's paddle velocities before colliding
    self.player.update_velocity()
    self.computer.update_velocity()

    self.puck.move()
    self.puck.bounce_off_walls(HEIGHT, MARGIN)

    # CHANGED (Task 1): pass the puck-centre y limits to the collision handler
    min_y = MARGIN + PUCK_RADIUS
    max_y = HEIGHT - MARGIN - PUCK_RADIUS
    handle_paddle_collision(self.puck, self.player, min_y, max_y)
    handle_paddle_collision(self.puck, self.computer, min_y, max_y)

    self._handle_goals()

def _handle_goals(self):
    # CHANGED (Task 2): a goal counts only when the puck is fully inside
    # the goal gap and has fully crossed the goal line.
    r = self.puck.radius
    in_gap = GOAL_TOP + r <= self.puck.y <= GOAL_BOTTOM - r

    if self.puck.x - r < MARGIN:
        if in_gap:
            if self.puck.x + r < MARGIN:

```

```

        # ADDED (Task 2): puck in the left goal -> computer scores
        self.computer_score += 1
        self._reset_puck()
    else:
        self.puck.x = MARGIN + r
        self.puck.vx = -self.puck.vx
    elif self.puck.x + r > WIDTH - MARGIN:
        if in_gap:
            if self.puck.x - r > WIDTH - MARGIN:
                # ADDED (Task 2): puck in the right goal -> player scores
                self.player_score += 1
                self._reset_puck()
            else:
                self.puck.x = WIDTH - MARGIN - r
                self.puck.vx = -self.puck.vx

def _reset_puck(self):
    self.puck.x, self.puck.y = WIDTH / 2, HEIGHT / 2
    self.puck.vx = 0
    self.puck.vy = 0

# ADDED (Task 2): result of the match based on the current scores
def winner_text(self):
    if self.player_score > self.computer_score:
        return "You win!"
    if self.computer_score > self.player_score:
        return "Computer wins!"
    return "Draw"

def draw(self, surface, font):
    from game import renderer
    renderer.draw_table(surface)
    renderer.draw_paddle(surface, self.player, renderer.COLOR_PLAYER)
    renderer.draw_paddle(surface, self.computer, renderer.COLOR_COMPUTER)
    renderer.draw_puck(surface, self.puck)

    # ADDED (Task 2): score display, one on each side of the centre line
    you_text = f"You {self.player_score}"
    cpu_text = f"Computer {self.computer_score}"
    text_y = MARGIN + 6
    renderer.draw_text(surface, font, you_text,
                       (WIDTH / 2 - 20 - font.size(you_text)[0], text_y),
                       renderer.COLOR_PLAYER)
    renderer.draw_text(surface, font, cpu_text,
                       (WIDTH / 2 + 20, text_y),
                       renderer.COLOR_COMPUTER)

    # ADDED (Task 2): winner banner, shown once the match is over (Task 3 sets this)
    if self.match_over:
        renderer.draw_banner(surface, font, self.winner_text())

```

Run `python main.py` and test three things: score into the right goal (expect "You 1"), let the computer score in the left goal (expect "Computer 1"), and hit the puck against the wall beside a goal gap (expect a bounce and no score). The puck freezes after a goal because Task 4 isn't done yet, so close the game and rerun it to test the other side. Note that the puck is scored only once it has fully disappeared into the gap.

If it behaves, commit:

```

cd [REDACTED PATH]
git add Lab_04/02_air_hockey/air-hockey/game/game_engine.py
git status --short
git commit -m "Task 2: implement match scoring"

```

`git status --short` should list only that one file as staged. Tell me what you see when testing, and I'll do Task 3 (the timer) next.

AADHAVAN

couple changes i made , to add to context, i uncollapsed Lab_04/02_air_hockey directory by bringing in README to same scope, and remove .gitignore (air hockey gitignore) from the repo

now task 2 changes have been made and updated in git, move on to next task ,

CLAUDE

Noted on the directory change. I don't know the new paths, so the git commands below use `git status` and `git add -u` instead of hardcoded paths. `-u` stages only tracked files you modified, so your `venv`, README moves and the leftover Lab_03 files stay out.

Task 3 (30-second timer) changes only `game/game_engine.py`. Timing uses `time.monotonic()` (real elapsed time) rather than counting frames, so the match lasts 30 seconds even if the frame rate wobbles. The countdown starts when the game engine is created, which is when the window opens. When it hits zero, `match_over` is set, the puck, AI and your paddle all freeze, and the winner banner appears ("Draw" on a tie). The timer is drawn at the bottom center, clear of the scores. `main.py` doesn't need to change.

Replace the whole `game/game_engine.py` with:

```
"""
GameEngine: owns the puck, both paddles, and the computer AI, and runs
one frame's worth of game logic.

Starter version: the puck bounces around and paddles can hit it, but
there is no scoring, no match timer, and the reset that happens after
a goal is incomplete. That's what Tasks 2-4 fix/add.
"""

import math # ADDED (Task 3): for rounding the countdown up
import random
import time # ADDED (Task 3): real-time clock for the match timer

from game.puck import Puck
from game.paddle import Paddle
from game.ai import ComputerAI
from game.collisions import handle_paddle_collision
from game.renderer import WIDTH, HEIGHT, MARGIN, GOAL_TOP, GOAL_BOTTOM

PLAYER_SPEED = 6
PUCK_RADIUS = 12
PADDLE_RADIUS = 28
INITIAL_PUCK_SPEED = 4.5
MATCH_DURATION = 30 # ADDED (Task 3): match length in seconds

class GameEngine:
    def __init__(self):
        self.puck = Puck(WIDTH / 2, HEIGHT / 2, PUCK_RADIUS)
        self._launch_puck()

        self.player = Paddle(
            x=WIDTH * 0.15, y=HEIGHT / 2, radius=PADDLE_RADIUS,
            min_x=MARGIN + PADDLE_RADIUS, max_x=WIDTH / 2 - PADDLE_RADIUS,
            min_y=MARGIN + PADDLE_RADIUS, max_y=HEIGHT - MARGIN - PADDLE_RADIUS,
        )
        self.computer = Paddle(
            x=WIDTH * 0.85, y=HEIGHT / 2, radius=PADDLE_RADIUS,
            min_x=WIDTH / 2 + PADDLE_RADIUS, max_x=WIDTH - MARGIN - PADDLE_RADIUS,
            min_y=MARGIN + PADDLE_RADIUS, max_y=HEIGHT - MARGIN - PADDLE_RADIUS,
```

```

)
self.ai = ComputerAI()

# ADDED (Task 2): match scores
self.player_score = 0
self.computer_score = 0
# ADDED (Task 2): True once the match timer runs out (set in Task 3)
self.match_over = False

# ADDED (Task 3): match timer state
self._start_time = time.monotonic()
self.time_left = float(MATCH_DURATION)

def _launch_puck(self):
    angle_choices = [0.3, 0.6, -0.3, -0.6]
    direction = random.choice([-1, 1])
    vy_factor = random.choice(angle_choices)
    self.puck.vx = INITIAL_PUCK_SPEED * direction
    self.puck.vy = INITIAL_PUCK_SPEED * vy_factor

def handle_input(self, keys_pressed):
    # ADDED (Task 3): ignore input once the match has ended
    if self.match_over:
        return
    import pygame
    dx = dy = 0
    if keys_pressed[pygame.K_UP]:
        dy -= PLAYER_SPEED
    if keys_pressed[pygame.K_DOWN]:
        dy += PLAYER_SPEED
    if keys_pressed[pygame.K_LEFT]:
        dx -= PLAYER_SPEED
    if keys_pressed[pygame.K_RIGHT]:
        dx += PLAYER_SPEED
    self.player.move_by(dx, dy)

def update(self):
    # ADDED (Task 3): stop all game logic once the match is over
    if self.match_over:
        return

    # ADDED (Task 3): update the countdown from real elapsed time
    elapsed = time.monotonic() - self._start_time
    self.time_left = max(0.0, MATCH_DURATION - elapsed)
    if self.time_left <= 0:
        self.match_over = True
        return

    self.ai.update(self.computer, self.puck)

    # ADDED (Task 1): record this frame's paddle velocities before colliding
    self.player.update_velocity()
    self.computer.update_velocity()

    self.puck.move()
    self.puck.bounce_off_walls(HEIGHT, MARGIN)

    # CHANGED (Task 1): pass the puck-centre y limits to the collision handler
    min_y = MARGIN + PUCK_RADIUS
    max_y = HEIGHT - MARGIN - PUCK_RADIUS
    handle_paddle_collision(self.puck, self.player, min_y, max_y)
    handle_paddle_collision(self.puck, self.computer, min_y, max_y)

    self._handle_goals()

def _handle_goals(self):
    # CHANGED (Task 2): a goal counts only when the puck is fully inside

```

```

# the goal gap and has fully crossed the goal line.
r = self.puck.radius
in_gap = GOAL_TOP + r <= self.puck.y <= GOAL_BOTTOM - r

if self.puck.x - r < MARGIN:
    if in_gap:
        if self.puck.x + r < MARGIN:
            # ADDED (Task 2): puck in the left goal -> computer scores
            self.computer_score += 1
            self._reset_puck()
        else:
            self.puck.x = MARGIN + r
            self.puck.vx = -self.puck.vx
    elif self.puck.x + r > WIDTH - MARGIN:
        if in_gap:
            if self.puck.x - r > WIDTH - MARGIN:
                # ADDED (Task 2): puck in the right goal -> player scores
                self.player_score += 1
                self._reset_puck()
            else:
                self.puck.x = WIDTH - MARGIN - r
                self.puck.vx = -self.puck.vx

def _reset_puck(self):
    self.puck.x, self.puck.y = WIDTH / 2, HEIGHT / 2
    self.puck.vx = 0
    self.puck.vy = 0

# ADDED (Task 2): result of the match based on the current scores
def winner_text(self):
    if self.player_score > self.computer_score:
        return "You win!"
    if self.computer_score > self.player_score:
        return "Computer wins!"
    return "Draw"

def draw(self, surface, font):
    from game import renderer
    renderer.draw_table(surface)
    renderer.draw_paddle(surface, self.player, renderer.COLOR_PLAYER)
    renderer.draw_paddle(surface, self.computer, renderer.COLOR_COMPUTER)
    renderer.draw_puck(surface, self.puck)

    # ADDED (Task 2): score display, one on each side of the centre line
    you_text = f"You {self.player_score}"
    cpu_text = f"Computer {self.computer_score}"
    text_y = MARGIN + 6
    renderer.draw_text(surface, font, you_text,
                       (WIDTH / 2 - 20 - font.size(you_text)[0], text_y),
                       renderer.COLOR_PLAYER)
    renderer.draw_text(surface, font, cpu_text,
                       (WIDTH / 2 + 20, text_y),
                       renderer.COLOR_COMPUTER)

    # ADDED (Task 3): remaining time, bottom centre
    timer_text = f"Time {math.ceil(self.time_left)}"
    renderer.draw_text(surface, font, timer_text,
                       (WIDTH / 2 - font.size(timer_text)[0] / 2,
                        HEIGHT - MARGIN - font.get_height() - 6))

    # ADDED (Task 2/3): winner banner, shown once the timer has run out
    if self.match_over:
        renderer.draw_banner(surface, font, self.winner_text())

```

Test it with a stopwatch: the display should count 30 down to 0 and freeze the game there with a banner. Leave one run idle to see "Draw" at 0-0. For a winner banner you need a goal, and since the

puck still freezes after a goal (Task 4), score once and let the clock run out. Check that the paddle and AI don't move after time-up.

Commit:

```
cd [REDACTED PATH]
git status --short
```

This should show exactly one line, **M** <path>/game/game_engine.py. If it shows anything else modified, stop and paste it to me. If it's clean, run:

```
git add -u Lab_04
git status --short
git commit -m "Task 3: add 30-second match timer"
```

The second **git status --short** should now show that file with **M** in the first column, meaning staged. Task 4 (puck reset) comes next. Let me know when this one is tested and committed.

AADHAVAN

done lets move on

CLAUDE

Task 4 (puck reset) changes only **game/game_engine.py**. The bug is that **_reset_puck** zeroes the velocity and never relaunches. The fix re-centers the puck and calls the existing **_launch_puck()**, which assigns a fresh velocity with a random direction and angle. Since **_launch_puck** overwrites both **vx** and **vy**, no velocity from the goal shot carries over. Two decisions I made, so tell me if you want either changed: the paddles stay where they are after a goal (only the puck resets), and the relaunch direction is random, not aimed at whoever was scored on.

Replace the whole **game/game_engine.py** with:

```
"""
GameEngine: owns the puck, both paddles, and the computer AI, and runs
one frame's worth of game logic.

Starter version: the puck bounces around and paddles can hit it, but
there is no scoring, no match timer, and the reset that happens after
a goal is incomplete. That's what Tasks 2-4 fix/add.
"""

import math # ADDED (Task 3): for rounding the countdown up
import random
import time # ADDED (Task 3): real-time clock for the match timer

from game.puck import Puck
from game.paddle import Paddle
from game.ai import ComputerAI
from game.collisions import handle_paddle_collision
from game.renderer import WIDTH, HEIGHT, MARGIN, GOAL_TOP, GOAL_BOTTOM

PLAYER_SPEED = 6
PUCK_RADIUS = 12
PADDLE_RADIUS = 28
INITIAL_PUCK_SPEED = 4.5
MATCH_DURATION = 30 # ADDED (Task 3): match length in seconds

class GameEngine:
    def __init__(self):
        self.puck = Puck(WIDTH / 2, HEIGHT / 2, PUCK_RADIUS)
        self._launch_puck()

        self.player = Paddle(
```

```

        x=WIDTH * 0.15, y=HEIGHT / 2, radius=PADDLE_RADIUS,
        min_x=MARGIN + PADDLE_RADIUS, max_x=WIDTH / 2 - PADDLE_RADIUS,
        min_y=MARGIN + PADDLE_RADIUS, max_y=HEIGHT - MARGIN - PADDLE_RADIUS,
    )
    self.computer = Paddle(
        x=WIDTH * 0.85, y=HEIGHT / 2, radius=PADDLE_RADIUS,
        min_x=WIDTH / 2 + PADDLE_RADIUS, max_x=WIDTH - MARGIN - PADDLE_RADIUS,
        min_y=MARGIN + PADDLE_RADIUS, max_y=HEIGHT - MARGIN - PADDLE_RADIUS,
    )
    self.ai = ComputerAI()

    # ADDED (Task 2): match scores
    self.player_score = 0
    self.computer_score = 0
    # ADDED (Task 2): True once the match timer runs out (set in Task 3)
    self.match_over = False

    # ADDED (Task 3): match timer state
    self._start_time = time.monotonic()
    self.time_left = float(MATCH_DURATION)

def _launch_puck(self):
    angle_choices = [0.3, 0.6, -0.3, -0.6]
    direction = random.choice([-1, 1])
    vy_factor = random.choice(angle_choices)
    self.puck.vx = INITIAL_PUCK_SPEED * direction
    self.puck.vy = INITIAL_PUCK_SPEED * vy_factor

def handle_input(self, keys_pressed):
    # ADDED (Task 3): ignore input once the match has ended
    if self.match_over:
        return
    import pygame
    dx = dy = 0
    if keys_pressed[pygame.K_UP]:
        dy -= PLAYER_SPEED
    if keys_pressed[pygame.K_DOWN]:
        dy += PLAYER_SPEED
    if keys_pressed[pygame.K_LEFT]:
        dx -= PLAYER_SPEED
    if keys_pressed[pygame.K_RIGHT]:
        dx += PLAYER_SPEED
    self.player.move_by(dx, dy)

def update(self):
    # ADDED (Task 3): stop all game logic once the match is over
    if self.match_over:
        return

    # ADDED (Task 3): update the countdown from real elapsed time
    elapsed = time.monotonic() - self._start_time
    self.time_left = max(0.0, MATCH_DURATION - elapsed)
    if self.time_left <= 0:
        self.match_over = True
        return

    self.ai.update(self.computer, self.puck)

    # ADDED (Task 1): record this frame's paddle velocities before colliding
    self.player.update_velocity()
    self.computer.update_velocity()

    self.puck.move()
    self.puck.bounce_off_walls(HEIGHT, MARGIN)

    # CHANGED (Task 1): pass the puck-centre y limits to the collision handler
    min_y = MARGIN + PUCK_RADIUS

```

```

max_y = HEIGHT - MARGIN - PUCK_RADIUS
handle_paddle_collision(self.puck, self.player, min_y, max_y)
handle_paddle_collision(self.puck, self.computer, min_y, max_y)

self._handle_goals()

def _handle_goals(self):
    # CHANGED (Task 2): a goal counts only when the puck is fully inside
    # the goal gap and has fully crossed the goal line.
    r = self.puck.radius
    in_gap = GOAL_TOP + r <= self.puck.y <= GOAL_BOTTOM - r

    if self.puck.x - r < MARGIN:
        if in_gap:
            if self.puck.x + r < MARGIN:
                # ADDED (Task 2): puck in the left goal -> computer scores
                self.computer_score += 1
                self._reset_puck()
            else:
                self.puck.x = MARGIN + r
                self.puck.vx = -self.puck.vx
        elif self.puck.x + r > WIDTH - MARGIN:
            if in_gap:
                if self.puck.x - r > WIDTH - MARGIN:
                    # ADDED (Task 2): puck in the right goal -> player scores
                    self.player_score += 1
                    self._reset_puck()
                else:
                    self.puck.x = WIDTH - MARGIN - r
                    self.puck.vx = -self.puck.vx

def _reset_puck(self):
    self.puck.x, self.puck.y = WIDTH / 2, HEIGHT / 2
    # CHANGED (Task 4): the old code set vx = vy = 0 here, which left the
    # puck frozen. Relaunch it instead: _launch_puck overwrites both
    # velocity components, so nothing from the goal shot carries over.
    self._launch_puck()

# ADDED (Task 2): result of the match based on the current scores
def winner_text(self):
    if self.player_score > self.computer_score:
        return "You win!"
    if self.computer_score > self.player_score:
        return "Computer wins!"
    return "Draw"

def draw(self, surface, font):
    from game import renderer
    renderer.draw_table(surface)
    renderer.draw_paddle(surface, self.player, renderer.COLOR_PLAYER)
    renderer.draw_paddle(surface, self.computer, renderer.COLOR_COMPUTER)
    renderer.draw_puck(surface, self.puck)

    # ADDED (Task 2): score display, one on each side of the centre line
    you_text = f"You {self.player_score}"
    cpu_text = f"Computer {self.computer_score}"
    text_y = MARGIN + 6
    renderer.draw_text(surface, font, you_text,
                       (WIDTH / 2 - 20 - font.size(you_text)[0], text_y),
                       renderer.COLOR_PLAYER)
    renderer.draw_text(surface, font, cpu_text,
                       (WIDTH / 2 + 20, text_y),
                       renderer.COLOR_COMPUTER)

    # ADDED (Task 3): remaining time, bottom centre
    timer_text = f"Time {math.ceil(self.time_left)}"
    renderer.draw_text(surface, font, timer_text,

```

```

(WIDTH / 2 - font.size(timer_text)[0] / 2,
HEIGHT - MARGIN - font.get_height() - 6))

# ADDED (Task 2/3): winner banner, shown once the timer has run out
if self.match_over:
    renderer.draw_banner(surface, font, self.winner_text())

```

Test by scoring several goals on both sides. After each one the puck should reappear at the center already moving, and the score should go up by exactly 1. It shouldn't score twice for one goal or ever sit still. Then let the clock run out and confirm the banner is right. Run the whole game once end to end, since this is the take you'll record.

Commit and push:

```

cd [REDACTED PATH]
git status --short

```

Expect exactly one modified file, `game_engine.py`. If anything else shows, paste it to me. If it's clean:

```

git add -u Lab_04
git status --short
git commit -m "Task 4: reset and relaunch puck after a goal"
git log --oneline -6
git push origin main

```

`git log` should show your 5 commits (starter, Tasks 1 to 4). Your untracked Lab_03 and **[REDACTED]** files stay untouched.

Two open items before the "after" video. First, the README says R restarts the game, but `main.py` has no handler for it. It's not in the four tasks, but the README lists it under controls, so a grader may test it. Adding it means a small `main.py` edit and a `reset_match()` method in the engine, as a sixth commit. Do you want it? Second, the after video needs to show a fixed collision, both scores, the timer, a goal followed by the puck relaunching, and the final banner. Ten seconds won't cover a full 30-second match, so tell me if you want a plan for what to capture. One option is a temporary 10-second `MATCH_DURATION` for the recording only, which you'd revert before the final commit, though I'd need you to confirm your faculty is fine with that.

AADHAVAN

lets do the change for "R" causing restart

do not worry about the after, i shall make an entire 30second video and trim out down to show proper all 4 bug fixes

after that final submission and review before submitting should be done, ill give you instructions for that, do not do it yourself, wait for my specific instruction

CLAUDE

Restarting means recreating the engine. `GameEngine()` already initializes the scores, timer, `match_over`, paddles, AI and puck, so no `game_engine.py` change is needed and this is a one-file commit. That replaces my earlier `reset_match()` idea, which would have duplicated the same setup code.

Replace the whole `main.py` with:

```

"""
Air Hockey (Lab Starter)

Run with:  python3 main.py

Controls: Arrow keys move your paddle (left side, blue).

```

```

        R restarts the match at any time.
"""

import pygame

from game.game_engine import GameEngine
from game.renderer import WINDOW_SIZE

def main():
    pygame.init()
    screen = pygame.display.set_mode(WINDOW_SIZE)
    pygame.display.set_caption("Air Hockey")
    clock = pygame.time.Clock()
    font = pygame.font.SysFont("consolas", 24)

    engine = GameEngine()
    running = True
    while running:
        for event in pygame.event.get():
            if event.type == pygame.QUIT:
                running = False
            # ADDED: press R at any time to restart (fresh scores, timer, puck, paddles)
            elif event.type == pygame.KEYDOWN and event.key == pygame.K_r:
                engine = GameEngine()

        keys = pygame.key.get_pressed()
        engine.handle_input(keys)
        engine.update()
        engine.draw(screen, font)

        pygame.display.flip()
        clock.tick(60)

    pygame.quit()

if __name__ == "__main__":
    main()

```

Test three cases. Press R mid-match and confirm the scores go to 0-0, the timer returns to 30, and the puck relaunches from the center. Press R after time runs out and confirm the banner disappears and play resumes. Press R several times quickly and confirm nothing breaks. One caveat: the timer is tied to the moment the engine is created, so the countdown restarts from 30 exactly at the press.

Commit and push:

```

cd [REDACTED PATH]
git status --short

```

Expect exactly one modified file, **main.py**. If anything else shows, paste it to me. If it's clean:

```

git add -u Lab_04
git status --short
git commit -m "Add R key to restart the match"
git push origin main

```

That makes 6 commits in total. I'll wait for your instructions on the final review and submission.

AADHAVAN

[Attached files: **[REDACTED FILE]** (image of the assignment instructions and deliverables page), README.md, main.py, __init__.py, ai.py, collisions.py, game_engine.py, paddle.py, puck.py, renderer.py]

some nested directories were refactored

some more commits were made, i also made a submissions directory to upload the final deliverables, once you verify all the files here as per the readme and instructions and conclude , ill take a pdf of this chat as well and finally commit and wrap it up

final path / structure

```
lab/                                     <-- Your main local repository root
├── .gitignore                         <-- Your root ignore file (untouched)
├── README.md
├── [REDACTED]/                       <-- Untracked documents folder
├── Lab_01/
├── Lab_02/
├── Lab_03/                           <-- Contains your architectural diagrams
└── Lab_04/                           <-- Fixed Lab 04 root folder
    ├── air-hockey/                   <-- Main application folder
    │   ├── .gitignore                <-- Local rules file (contains: venv/)
    │   ├── main.py                   <-- Application entry point
    │   ├── README.md                 <-- Lab documentation
    │   ├── requirements.txt           <-- Pygame package dependencies
    │   ├── venv/                     <-- Local Python environment (safely ignored)
    │   └── game/                     <-- Core engine module
    │       ├── __init__.py
    │       ├── ai.py
    │       ├── collisions.py
    │       ├── game_engine.py <-- Your match scoring updates are here
    │       ├── paddle.py
    │       ├── puck.py
    │       └── renderer.py
    └── submissions/                  <-- Target directory for your final deliverables
```

readme and instructions given, (for reference)

return review with green ticks

do not worry about cosmetic issues / visual cues, only raise issues if something deviates functionality wise or from the expected submission format wise

keep answer short and informative

CLAUDE

- ✓ Task 1: collision now pushes the puck out of the paddle, reflects about the contact normal, uses paddle velocity, and caps speed at 14, so no sticking or tunneling.
- ✓ Task 2: a goal scores only when the puck is fully inside the gap and past the line, and both scores are displayed. Left goal credits the computer, right goal credits you. Wall hits beside the gap bounce.
- ✓ Task 3: the timer is real-time (`time.monotonic`), counts down from 30, freezes the puck, AI and player at 0, and shows "You win!", "Computer wins!" or "Draw".
- ✓ Task 4: `_reset_puck` re-centers the puck and calls `_launch_puck`, which overwrites both velocity components, so the puck is never motionless.
- ✓ R restarts (README Controls): a new `GameEngine()` resets scores, timer, `match_over`, puck and paddles.
- ✓ Structure matches the README folder structure, with `game/` holding the six modules plus `__init__.py`. I can only verify the code files uploaded and the tree you pasted, not what git actually tracks.
- ✓ Step 10: you never pushed to SETAPESU26, so no PR issue.

✓ Python 3.9.6 is fine, since nothing in the code needs 3.10+.

Things to check or decide:

1. **Residual edge case.** If your paddle pins the puck against an end wall outside the goal gap, the puck can still jitter, because `_handle_goals` flips `vx` unconditionally. The corner-slide fix only covers top and bottom walls. It's rare and unlikely to show up in normal play or your video. I'd leave it, but I can fix it as another commit if you want.
2. **Chat deliverable.** The README asks for a chat link and the instructions ask for a PDF/doc export. Submit both.
3. **Repo hygiene.** Your local `.gitignore` only contains `venv/`, and you ran the game before the first commit. That means `__pycache__` or `.DS_Store` may have been committed. Run:

```
cd [REDACTED PATH]
git ls-files Lab_04 | grep -E "__pycache__|\.DS_Store|venv/"
```

There should be no output. If lines appear, paste them and I'll give the exact removal commands.

4. **Final commit.** Check `ls -lh Lab_04/submissions` first. Each file must be under 100 MB, which is GitHub's push limit, so trim the 30-second video. Then:

```
git status --short
git add Lab_04/submissions
git commit -m "Add Lab_04 submissions: videos and chat history"
git push origin main
git status -sb
```

`git status --short` should show only files in `Lab_04/submissions`. The last command should show no "ahead" count. Open the repo on GitHub afterward to confirm the videos and PDF are there.

5. **Folder name.** The instructions say `Lab-4`, and you're using `Lab_04` to match `Lab_01` to `Lab_03`. That's fine, but it's a literal mismatch if a grader looks for the exact name.
6. **PDF export.** Export after this reply so the final review is included. Use `Cmd+P`, then `Save as PDF`, and check that long code blocks aren't cut off.

AADHAVAN

all checked and verified

now give a pdf giving full transcript of our conversation (my exact prompts and your exact replies)
separately list out possible sensitive information i might have shared accidentally

Note: Claude's reply to the final message above is this PDF itself and is not part of the transcript.